CS683 PA2 – Task 2A: Instruction-Conscious L2 Replacement (I-RRIP)

Team: TeamName (LDAP1, LDAP2) | Simulator: ChampSim | 25M warmup + 25M simulated instructions per trace

1. Summary of the work

> **What was built.** An instruction-conscious replacement policy for the L2 cache (`replacement/irrip.l2c_repl`), derived from 2-bit SRRIP. The policy receives an `isInstruction` flag with every insertion/promotion and favours instruction lines in insertion, promotion and eviction.
>
> **Headline result.** L2 instruction load MPKI decreased on all three traces. On the instruction-dominated Trace2 it fell from 18.85 to 13.26 (-30%), total L2C load MPKI fell by 17% and IPC improved by 2.1% (speedup 1.021). Trace1 is neutral and Trace3 flat. The price is higher L2 *data* load MPKI on every trace, which is the motivation for the dynamic policy of Task 2B.

2. Work log (what was done, in order)

1. **Build environment.** The first build (Ubuntu 22.04, default GCC 11.4) compiled but crashed at run time (segmentation fault inside `O3_CPU::execute_instruction` with an invalid `this` pointer). The build log showed the warning *"control reaches end of non-void function"* for `PAGE_TABLE_WALKER::handle_page_fault` in `src/page_table_walker.cc`: the function is declared to return `uint64_t` but has no `return`. This is undefined behaviour that older compilers (the README asks for GCC 7) tolerated and GCC 11 does not. All three call sites ignore the return value, so a `return 0;` was added at the end of the function (no change in simulated behaviour), and the simulator is built with GCC 9 (`CXX=g++-9 ./build_champsim.sh ...`).
2. **Baseline.** LRU L2C, no prefetcher, run on Trace1–3 (25M warmup, 25M simulation). The trace files are given as `.gz`, which ChampSim reads directly.
3. **Plumbing.** `update_replacement_state` was extended with a final `uint8_t isInstruction` parameter: function-pointer type and declarations in `inc/cache.h`; definitions in `base_replacement.cc` and all `lru.*_repl` files and `rrip.l2c_repl`; 8 call sites in `src/cache.cc` (passing the packet's `.instruction` field from the MSHR/WQ/RQ/PQ entry) and 3 BTB call sites in `src/ooo_cpu.cc` (passing 0). **Verification:** the LRU baseline was rebuilt after the change and re-run; IPC and all L2C load MPKI lines were identical to the pre-change run on all three traces.
4. **I-RRIP.** Implemented in one file with a compile-time switch `IRRIP_VERSION` (1, 2, 3), so every change could be measured on its own.
5. **Experiments.** LRU, plain RRIP, and I-RRIP V1/V2/V3 on all three traces; results below. A helper script (`run_ladder.sh`) rebuilt and ran each version.

3. The policy and its versions

Background: RRIP keeps a 2-bit re-reference prediction value (RRPV) per line: 0 = expected to be reused soon, 3 = evict next. SRRIP inserts at RRPV 2, promotes a hit line to 0, and evicts a line at RRPV 3 (ageing all lines when none is at 3). Server-style code does not fit in L1I, so L2 hits for instruction fetches matter for front-end throughput; I-RRIP gives instruction lines preferential treatment. The three versions are cumulative.

Version	Insertion (fill)	Promotion (hit)	Eviction (victim choice)
RRIP (reference)	all lines at RRPV 2	all hits to RRPV 0	first line at RRPV 3, age if none
V1	**instruction at RRPV 0**, data at RRPV 2	unchanged (to 0)	unchanged
V2	as V1	**instruction hit to 0; data hit decremented by one step**	unchanged
V3 (final)	as V1	as V2	**among lines at RRPV 3, evict a data line first**; otherwise any line at 3; age if none

Why each change

Insertion (V1). A line brought in by an instruction fetch is placed at the most-protected position (RRPV 0), so it survives longer than the data fills that compete for the same set. Data fills keep the baseline RRPV 2 ("long re-reference" prediction), so streaming data still ages out quickly and is not given extra protection. This is the cheapest change and is expected to give most of the instruction-miss reduction.

Promotion (V2). An instruction line that is hit again is promoted fully (to 0), while a data line hit moves down only one step. A data line therefore needs repeated reuse to reach the protected state, and instruction lines that proved useful are hard to displace. The risk is that genuinely reused data lines are protected less well.

Eviction (V3). When several lines are at the eviction value, a data line is chosen before an instruction line (read from `block[set][way].instruction`, which the cache sets at fill time). If no data line is at RRPV 3 the normal RRIP search and ageing loop is used, so the ageing mechanism is unchanged. This only changes the tie-break, which keeps the modification small and easy to reason about.

Implementation notes. The policy uses only the existing 2-bit RRPV per line plus the per-block instruction bit already present in the cache, i.e. no additional storage. Prefetch packets are not treated specially here (that is Task 3).

4. Results

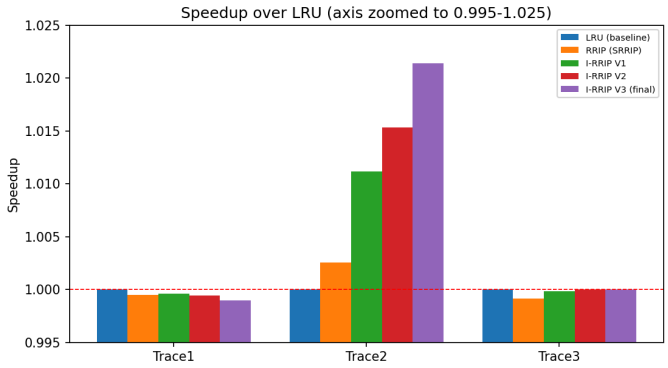
All numbers are from the ROI of each run (25M instructions after a 25M warmup). Speedup = IPC / IPC of LRU. MPKI values are L2C *load* misses per 1000 instructions; "instr" and "data" split the load misses by request type.

Trace	Policy	IPC	Speedup vs LRU	L2C load MPKI	L2 data load MPKI	L2 instr load MPKI
Trace1	LRU (baseline)	0.232345	1.0000	17.696	7.358	10.338
Trace1	RRIP (SRRIP)	0.232231	0.9995	17.639	7.077	10.562
Trace1	I-RRIP V1	0.232256	0.9996	17.682	7.567	10.115
Trace1	I-RRIP V2	0.232223	0.9995	17.695	7.722	9.973
Trace1	I-RRIP V3 (final)	0.232109	0.9990	17.753	7.902	9.850
Trace2	LRU (baseline)	0.251158	1.0000	27.357	8.504	18.853
Trace2	RRIP (SRRIP)	0.251806	1.0026	25.746	7.965	17.781
Trace2	I-RRIP V1	0.253964	1.0112	24.848	8.732	16.116
Trace2	I-RRIP V2	0.255011	1.0153	23.894	9.144	14.750
Trace2	I-RRIP V3 (final)	0.256535	1.0214	22.602	9.344	13.258
Trace3	LRU (baseline)	0.382065	1.0000	3.954	2.830	1.124
Trace3	RRIP (SRRIP)	0.381739	0.9991	4.039	2.842	1.197
Trace3	I-RRIP V1	0.382002	0.9998	3.948	2.871	1.077
Trace3	I-RRIP V2	0.382079	1.0000	3.896	2.871	1.024
Trace3	I-RRIP V3 (final)	0.382079	1.0000	3.878	2.883	0.995

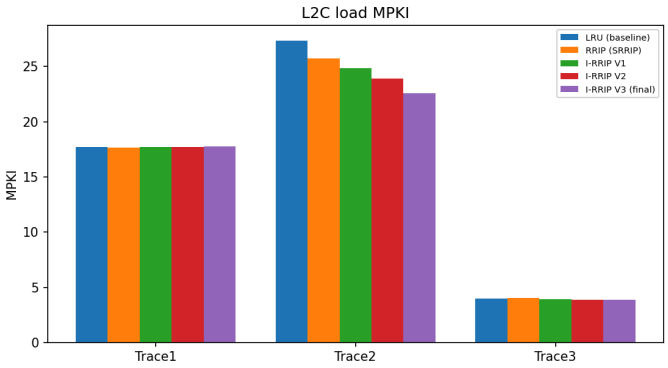
Mean speedup over LRU

Policy	Trace1	Trace2	Trace3	Mean
LRU (baseline)	1.0000	1.0000	1.0000	1.0000
RRIP (SRRIP)	0.9995	1.0026	0.9991	1.0004
I-RRIP V1	0.9996	1.0112	0.9998	1.0035
I-RRIP V2	0.9995	1.0153	1.0000	1.0050
I-RRIP V3 (final)	0.9990	1.0214	1.0000	1.0068

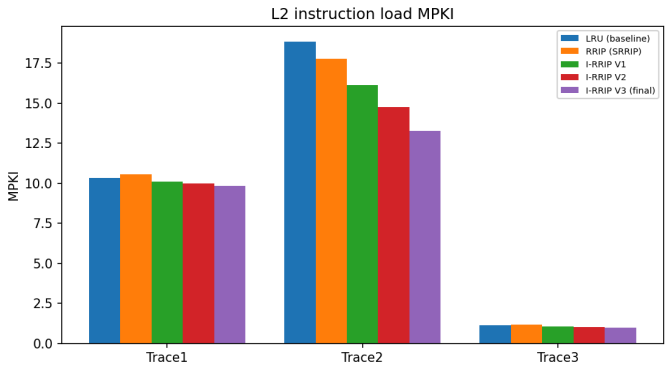
Plots



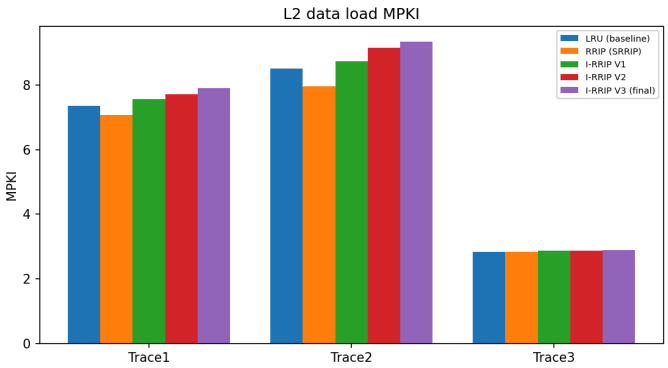
Speedup over LRU (axis zoomed to 0.995-1.025)



L2C load MPKI



L2 instruction load MPKI



L2 data load MPKI

I-RRIP V3 compared with LRU

Trace	Speedup	L2 instr load MPKI	L2 data load MPKI	L2C load MPKI
Trace1	0.9990	10.338 -> 9.850 (-4.7%)	7.358 -> 7.902 (+7.4%)	17.696 -> 17.753 (+0.3%)
Trace2	1.0214	18.853 -> 13.258 (-29.7%)	8.504 -> 9.344 (+9.9%)	27.357 -> 22.602 (-17.4%)
Trace3	1.0000	1.124 -> 0.995 (-11.5%)	2.830 -> 2.883 (+1.9%)	3.954 -> 3.878 (-1.9%)

I-RRIP V3 compared with plain RRIP

Trace	Speedup	L2 instr load MPKI	L2 data load MPKI	L2C load MPKI
Trace1	0.9995	10.562 -> 9.850 (-6.7%)	7.077 -> 7.902 (+11.7%)	17.639 -> 17.753 (+0.6%)
Trace2	1.0188	17.781 -> 13.258 (-25.4%)	7.965 -> 9.344 (+17.3%)	25.746 -> 22.602 (-12.2%)
Trace3	1.0009	1.197 -> 0.995 (-16.9%)	2.842 -> 2.883 (+1.4%)	4.039 -> 3.878 (-4.0%)

Contribution of each change (marginal, per step)

"Speedup change" is in percentage points of the LRU IPC; the last column is how many extra data load misses (MPKI) each step adds per instruction load miss (MPKI) it removes.

Trace	Step	Speedup change	dInstr MPKI	dData MPKI	Data misses added per instr miss saved
Trace1	Insertion (LRU -> V1)	-0.04 pts	-0.223	+0.209	0.94
Trace1	+ Promotion (V1 -> V2)	-0.01 pts	-0.142	+0.155	1.09
Trace1	+ Eviction (V2 -> V3)	-0.05 pts	-0.123	+0.180	1.47
Trace2	Insertion (LRU -> V1)	+1.12 pts	-2.738	+0.228	0.08
Trace2	+ Promotion (V1 -> V2)	+0.42 pts	-1.366	+0.412	0.30
Trace2	+ Eviction (V2 -> V3)	+0.61 pts	-1.491	+0.200	0.13
Trace3	Insertion (LRU -> V1)	-0.02 pts	-0.047	+0.041	0.87
Trace3	+ Promotion (V1 -> V2)	+0.02 pts	-0.052	-0.000	-0.00
Trace3	+ Eviction (V2 -> V3)	+0.00 pts	-0.029	+0.012	0.41

5. Explanation of the results

Trace2 – large gain. This trace is instruction-bound: its L1I MPKI is about 40 and instruction loads cause roughly two thirds of L2 load misses under LRU. Every step helps: V1 / V2 / V3 give speedups of 1.0112 / 1.0153 / 1.0214. The code footprint also exceeds the L2 (under LRU, instruction fills evict other instruction lines about 300 thousand times), so protecting instruction lines changes which code survives and substantially reduces instruction misses. Against plain RRIP the gain is larger still (speedup 1.0188 over RRIP, instruction MPKI -25.4%), showing the improvement comes from the instruction-aware changes and not from using RRIP.

Trace1 – neutral. Instruction misses fall at every step (MPKI 10.34 to 9.85), but data misses rise by almost the same amount (MPKI 7.36 to 7.90), so total L2C load MPKI and IPC stay flat (speedup 0.9990). This suggests that instruction and data lines compete about equally for the L2 on this trace, so favouring one side mostly moves misses from one type to the other (an interpretation of the numbers; it was not tested separately).

Trace3 – flat. Instruction pressure is low (L1I MPKI about 2.1, L2 instruction MPKI about 1.1), so there is little for an instruction-aware policy to gain; instruction MPKI still falls (MPKI 1.12 to 1.00) while data MPKI rises slightly, and IPC is unchanged.

Cost-effectiveness of the three changes. On Trace2, insertion removes the most instruction misses for the fewest added data misses (about 0.08 per instruction miss saved); eviction costs 0.13 and promotion 0.30. Promotion is therefore the least cost-effective change, most likely because slowing data promotion reduces the protection of reused data lines (not tested in isolation). V3 nevertheless has the best mean speedup (1.0035 for V1, 1.0050 for V2, 1.0068 for V3) and the best result on the trace where the policy matters most.

6. Why this design

- The task goal is to reduce L2 instruction load MPKI. V3 achieves the lowest instruction MPKI on every trace among the variants tested, and the highest speedup on Trace2 and highest mean speedup.
- It is simple and cheap: no extra state beyond what the cache already holds, and RRIP's ageing logic is preserved.
- The incremental ladder gives evidence for each decision rather than assuming all three changes are needed; it also shows which one is least efficient (promotion).
- The data-MPKI increase and the neutral Trace1 result are the honest limits of a static policy: it helps when instruction pressure is high and costs data misses when it is not. A policy that switches itself on only when instruction pressure is high (set dueling, Task 2B) is the natural fix.

7. Limitations and possible follow-ups

- Gains are modest (at most about 2%) and measured on one 25M-instruction window per trace, without variance estimates.
- Untested variants: instruction insertion at RRPV 1 instead of 0 (less aggressive), leaving data promotion unchanged while keeping the eviction change, or limiting instruction protection per set.

- Only three traces were used; behaviour on other workloads (especially data-heavy ones, like Trace3) is expected to be neutral-to-negative for a static policy.

8. How to reproduce

```
# build (GCC 9) and run, e.g. I-RRIP V3
CXX=g++-9 ./build_champsim.sh irrip
./bin/hashed_perceptron-no-no-no-no-no-no-no-lru-lru-lru-irrip-lru-lru-lru-lru-1core-irrip \
    -warmup_instructions 25000000 -simulation_instructions 25000000 -traces ~/traces/Trace1.gz >
results/irrip_v3_Trace1.txt
# IRRIP_VERSION (1, 2 or 3) is the #define at the top of replacement/irrip.l2c_repl; run_ladder.sh <v>
rebuilds and runs all traces
# baseline: CXX=g++-9 ./build_champsim.sh baseline ;  plain RRIP: CXX=g++-9 ./build_champsim.sh rrip
python3 make_summary.py     # regenerates tables, plots and this document
```

Files: `replacement/irrip.l2c_repl` (policy), `inc/cache.h`, `src/cache.cc`, `src/ooo_cpu.cc`, `replacement/*.l2c_repl` and other policy files (signature change), `src/page_table_walker.cc` (missing return fix), `results/` (raw ChampSim outputs, table, plots), `run3.sh`, `run_ladder.sh`, `make_summary.py`.